Alternatively, you can ensure that the thread has completed execution before the function exits by *joining* with the thread.

2.1.2. Waiting for a thread to complete

If you need to wait for a thread to complete, you can do this by calling `join()` on the associated `std::thread` instance. In the case of listing 2.1, replacing the call to `my_thread.detach()` before the closing brace of the function body with a call to `my_thread.join()` would therefore be sufficient to ensure that the thread was finished before the function was exited and thus before the local variables were destroyed. In this case, it would mean there was little point running the function on a separate thread, because the first thread wouldn't be doing anything useful in the meantime, but in real code the original thread would either have work to do itself or it would have launched several threads to do useful work before waiting for all of them to complete.

`join()` is simple and brute force—either you wait for a thread to finish or you don't. If you need more fine-grained control over waiting for a thread, such as to check whether a thread is finished, or to wait only a certain period of time, then you have to use alternative mechanisms such as condition variables and futures, which we'll look at in chapter 4. The act of calling `join()` also cleans up any storage associated with the thread, so the `std::thread` object is no longer associated with the now-finished thread; it isn't associated with any thread. This means that you can call `join()` only once for a given thread; once you've called `join()`, the `std::thread` object is no longer joinable, and `joinable()` will return `false`.

2.1.3. Waiting in exceptional circumstances

As mentioned earlier, you need to ensure that you've called either `join()` or `detach()` before a `std::thread` object is destroyed. If you're detaching a thread, you can usually call `detach()` immediately after the thread has been started, so this isn't a problem. But if you're intending to wait for the thread, you need to pick carefully the place in the code where